

SCREENBITES CINEMA

Documentación Técnica y de Proyecto



Autores del Proyecto: Enrique y Alejandro

Tecnologías: Laravel, PHP, MySQL, JavaScript

Año: 2026

Índice de Contenidos

1. Introducción y Finalidades del Proyecto
2. Arquitectura del Sistema: Laravel, WordPress y MySQL
3. Análisis Inicial: Gestión de Usuarios, Roles y Permisos
4. Diseño de Interfaz y Experiencia de Usuario (UI/UX)
5. Sistema de Reservas y Compra (Carrito y Checkout)
6. Gestión de Salas, Sesiones y Sincronización Temporal
7. Panel de Perfil de Usuario y Gestión de Tickets
8. Sistema de Eventos Especiales y Lógica Condicional
9. Sistema de Moderación de Reseñas y Panel Admin
10. Seguridad, Validaciones y Control de Errores
11. Despliegue de la Aplicación en Entornos de Producción
12. Bibliografía y Recursos Consultados

1. Introducción y Finalidades del Proyecto

El presente proyecto consiste en el desarrollo integral de una aplicación web para la gestión, reserva y venta de entradas de un cine premium denominado "**Screenbites**".

Las finalidades principales que han guiado el desarrollo arquitectónico y visual de este proyecto son:

- **Modernizar la experiencia de usuario:** Ofrecer una navegación fluida, intuitiva y visualmente atractiva (estilo dark mode) que se aleje de las plataformas de cine tradicionales, integrando elementos dinámicos interactivos.
- **Gestión unificada de productos:** Permitir a los usuarios no solo comprar entradas, sino añadir productos de restauración (comida, bebida y combos exclusivos) en un mismo flujo de compra ininterrumpido.
- **Escalabilidad técnica:** Crear un sistema robusto mediante el framework Laravel (MVC) que soporte múltiples salas, distintas sesiones automatizadas, y la gestión de eventos especiales con lógicas condicionales exclusivas.
- **Autogestión y Roles:** Dotar al usuario de un panel de control completo para gestionar su cuenta y sus tickets, al mismo tiempo que se provee a la administración de herramientas automatizadas para la moderación de la plataforma.

2. Arquitectura del Sistema: Laravel, WordPress y MySQL

Para la construcción de la plataforma se ha optado por un ecosistema mixto, donde cada tecnología cumple una función específica orientada al rendimiento y la seguridad.

El Motor Principal: Laravel (PHP)

Laravel actúa como el núcleo (backend) de la aplicación bajo el patrón Modelo-Vista-Controlador (MVC). Se encarga del enrutamiento seguro (`web.php`), el control de la lógica de negocio (`BookingController` , `AdminController` , `MovieController`), la validación de formularios y el renderizado dinámico de las vistas mediante el motor **Blade**.

Gestor de Base de Datos: phpMyAdmin / MySQL

Todos los datos relacionales (usuarios, sesiones, reservas y reseñas) se almacenan en una base de datos MySQL, administrada visualmente a través de phpMyAdmin. Esto permite ejecutar consultas complejas (mediante Eloquent ORM y Query Builder) para cruzar datos, como comprobar qué butacas están ocupadas en una sesión específica.

La evolución respecto a WordPress

En las fases iniciales del proyecto, se contempló el uso de WordPress para la gestión de contenido rápido (como el sistema de comentarios utilizando la tabla `wp_comments`). Sin embargo, conforme el proyecto escaló, se determinó que **depender de una base de datos externa limitaba la seguridad y la integridad relacional**.

Decisión Arquitectónica: Se diseñó una migración nativa en Laravel para crear la tabla `reviews` . Esto permitió desconectar los comentarios de WordPress y vincularlos directamente a los usuarios registrados en Laravel, posibilitando el sistema de puntuación por estrellas y los estados de moderación (Pending/Approved) de forma nativa e independiente. Tras ello, se realizó una purga (truncate) de datos residuales en WordPress.

3. Análisis Inicial: Gestión de Usuarios, Roles y Permisos

El sistema cuenta con un control de acceso basado en roles (RBAC) con cuatro niveles de interacción claramente diferenciados:

- **Usuario No Registrado (Guest):** Tiene acceso de lectura a la cartelera, menús y equipo. Al intentar comprar un ticket o dejar una reseña, la interfaz bloquea la acción y muestra un Call to Action dinámico (`LOGIN TO BUY`) que redirige al sistema de autenticación.
- **Usuario Estándar:** Usuario autenticado. Puede realizar compras, acceder a su panel de perfil, ver su historial de tickets y publicar reseñas (las cuales nacen con estado "Pendiente de Aprobación").
- **Usuario VIP:** Un estatus premium que otorga beneficios económicos. El sistema detecta su rol en el momento de la compra (Checkout) y, mediante cálculo de backend y JavaScript, inyecta automáticamente un **descuento del 10%** en el desglose final de tickets y menús adquiridos.
- **Administrador:** Acceso exclusivo mediante credenciales protegidas. Dispone de un Panel de Administración (`admin.index`) desde donde puede banear usuarios, modificar roles (ascender a VIP/Admin) y moderar todas las reseñas de la plataforma.

Se ha implementado un **Modo Demo** en las credenciales por defecto (admin@admin.com / admin123456) integradas mediante *Seeders* de Laravel, garantizando que el tribunal evaluador pueda acceder a todas las funcionalidades sin requerir un registro manual.

4. Diseño de Interfaz y Experiencia de Usuario (UI/UX)

La plataforma "Screenbites" sigue los estándares modernos de las aplicaciones VOD (Video on Demand) con una estética oscura (Dark Mode) que resalta el contenido audiovisual y los colores de llamada a la acción (Call to Action).

Colorimetría Dinámica

A través de variables CSS inyectadas de forma dinámica desde los controladores (`var(--color-principal)`), la interfaz de compra adapta sus colores de acento, bordes y botones basándose en la paleta de colores del póster de la película seleccionada, generando una experiencia inmersiva única por película.

Componentes Interactivos Clave

- **Sección Team:** Un área de presentación ("Meet The Team") maquetada con tarjetas interactivas que revelan redes sociales y roles al pasar el cursor (efecto hover), aportando profesionalidad al producto y detallando a los desarrolladores.
- **Modales de Tráiler (Iframe):** Se ha implementado un sistema JavaScript puro en `pelicula.blade.php` que extrae el ID de cualquier enlace de YouTube mediante Expresiones Regulares (Regex) y genera un reproductor emergente superpuesto con auto-reproducción (`autoplay=1`).
- **Carrusel de Imágenes:** Componentes de arrastre sin dependencias de terceros para mostrar fotogramas de las películas (Movie Stills).

5. Sistema de Reservas y Compra (Carrito y Checkout)

El núcleo transaccional del cine es un flujo asíncrono y multipartes diseñado para evitar pérdidas de datos en el cliente.

Memoria de Carrito Inteligente (LocalStorage por Usuario)

El carrito se gestiona en el cliente mediante la API Web `localStorage`, pero se le ha añadido una capa superior de lógica: **la clave de almacenamiento se concatena con el ID del usuario actual de Laravel** (`screenbites_cart_{{ Auth::id() }}`). Esto garantiza que si el usuario cierra sesión, el carrito se guarde en su perfil y no aparezca a otro usuario en la misma terminal.

Persistencia y Memoria de Navegación

Durante la compra, si el usuario retrocede desde la ventana de menú (`food.blade.php`) a la de selección de butacas (`booking.blade.php`), un script lee el array del carrito y **vuelve a marcar visualmente las butacas que ya había elegido**. Esto evita la frustración del usuario en la re-selección manual de los asientos.

Combos Exclusivos Condicionales y Rediseño de Carrito

En la sección de restauración, el sistema compara el `movie_id` actual. Si coincide con eventos guardados, renderiza un apartado de "Combos Exclusivos" (ej. Cubo amarillo de Kill Bill) que solo son accesibles y comprables para la película correspondiente.

Se realizó un rediseño completo de la interfaz visual del carrito para desglosar claramente los asientos, la comida y el descuento VIP (10%) antes del pase al Checkout.

6. Gestión de Salas, Sesiones y Sincronización Temporal

Algoritmo de Auto-Sincronización de Fechas (Cine Eterno)

Para asegurar que la aplicación siempre tenga contenido disponible independientemente del día de corrección o visualización de la entrega, se desarrolló un algoritmo de auto-sincronización en el archivo `BookingController.php`.

Al acceder a la pantalla de reserva, el sistema verifica si existen sesiones caducadas. De ser así, se determina el diferencial de días (`diffInDays`) entre la fecha de base de datos y la fecha actual real. Acto seguido, una consulta SQL con `DATE_ADD` traslada automáticamente el calendario del cine al tiempo presente, purga las butacas antiguas (`truncate`) y deja las salas listas para la compra en vivo.

Control de Aforo y Modales de Restricción

Las butacas compradas quedan inhabilitadas visual y funcionalmente (gris y sin evento de clic). Asimismo, un controlador en JavaScript interrumpe la selección si el cliente marca **más de 8 butacas simultáneamente**, invocando un modal informativo sobre reservas masivas por política de aforo.

7. Panel de Perfil de Usuario y Gestión de Tickets

Seguridad en el Cambio de Contraseña

En el perfil del usuario (`profile.blade.php`), el cambio de contraseñas es gestionado mediante eventos de validación *real-time* en el cliente, apoyado con seguridad Backend.

Mediante Expresiones Regulares, se evalúa que la contraseña introducida contenga un mínimo de 8 caracteres, uso de mayúsculas/minúsculas, inclusión de números y símbolos. Los

requisitos se colorean de verde al cumplirse, y el botón de guardado (Submit) permanece deshabilitado hasta que las contraseñas coinciden y superan los test descritos, actualizando el *hash* de la base de datos a continuación.

Motor de Expiración de Tickets Dinámico

Dentro del historial de compras ("My Tickets"), el sistema discrimina el estado de un pase utilizando la librería oficial **Carbon** de Laravel. No se limita al transcurso del día de la reserva; se suma la duración de la película (e.g., 2 horas) a su hora de inicio registrada:

```
$sessionDateTime = \Carbon\Carbon::parse($dateToParse . ' ' . $timeToParse);  
$isExpired = $sessionDateTime->addHours(2)->isPast();
```

Si la evaluación devuelve *True*, el elemento adquiere el tag de "EXPIRED", perdiendo color (grayscale) en el panel.

8. Sistema de Eventos Especiales y Lógica Condicional

Se ha implementado la gestión condicional para eventos especiales del cine, ejemplificada con el concepto del **"Blind Ticket" (Cine a Ciegas)** en la pestaña de *Events*.

- **Bloqueo Activo de Duplicados en Carrito:** El sistema evalúa el carrito almacenado. Si el Blind Ticket está presente e intentan reservarlo nuevamente desde otra página, un Modal irrumpe la pantalla informando al cliente que se trata de un "Asiento Único" intransferible.
- **Evaluación de Propiedad (DB exists):** Tras el Checkout exitoso, la vista Blade lanza un `DB::table('bookings')->where(...)->exists()`. Si la compra es afirmativa, los botones de compra se inhabilitan permanentemente y el indicativo de estado muta a "TICKET CLAIMED - ALREADY PURCHASED".
- **Animación de Revelación del Evento (Checkout):** Al efectuar el pago de este evento especial, el panel de *Success* convencional es sustituido temporalmente. Se ejecuta un modal negro con animaciones de "glitch" en CSS sobre un icono de interrogación, y pasados tres segundos de carga en JavaScript, un "flashazo" blanco desvela de forma inmersiva el póster y la sinopsis de la película de terror retro seleccionada.

9. Sistema de Moderación de Reseñas y Panel Admin

Uno de los logros de la plataforma es su sistema nativo de gestión de calidad en los comentarios y valoraciones del cine.

Flujo de Reseñas en Vistas de Película

Al completarse el traslado desde WordPress, el envío de reseñas (por un usuario registrado) almacena un registro en la tabla de Laravel con un valor restrictivo predeterminado: `'status'`
`=> 'pending'`.

Las consultas (queries) a la base de datos están configuradas para ocultar cualquier comentario que no tenga el estado `approved`. El autor original, no obstante, sí logra ver sus reseñas pendientes junto a un *badge naranja* (*PENDING APPROVAL*) a modo de trazabilidad personal.

Flujo de Moderación en Panel de Administración

Desde `admin/index.blade.php`, el administrador despliega una vista global con una relación `JOIN` entre la tabla de reseñas y los usuarios autores reales. Desde aquí, a través de peticiones HTTP enrutadas (`PATCH` y `DELETE`), el administrador puede transicionar el estado de un comentario de Pendiente a Aprobado para publicarlo al mundo, o ejecutar su borrado definitivo si incumple normas de la comunidad.

10. Seguridad, Validaciones y Control de Errores

El código general ha sido reforzado para proteger las ejecuciones del servidor:

- **Validaciones Backend en Pagos:** El controlador `processPayment` evalúa la autenticidad y el formato (e.g., regex numéricos, longitud obligatoria 16/19, validación lógica de fecha expirada de tarjeta de crédito) antes de volcar la tabla de compras a Base de Datos.
- **Directivas CSRF e Inyecciones:** Los formularios de envío (cambio de rol, compra de comida, reseñas) implementan tokens directivos `@csrf` de Laravel como protección pasiva para evitar falsificaciones de peticiones en sitios cruzados.
- **Captura de Rutas Inválidas:** El enrutamiento de peticiones de una sesión inexistente (``abort(404)``) devuelve vistas controladas con la temática de diseño de la marca "Screenbites".

11. Despliegue de la Aplicación en Entornos de Producción

Como consideración técnica final, se ha evaluado la escalabilidad de la arquitectura actual para su posterior despliegue global. Debido al empaquetamiento mixto del código, las metodologías recomendadas para su publicación real en Internet exigen un entorno LEMP/LAMP:

1. **Hosting Compartido o cPanel:** Configurando los puntos de enrutamiento al directorio `/public` de Laravel, permitiendo hospedar tanto los archivos del Framework como exportar la base de datos SQL alojada actualmente en el phpMyAdmin en local.
2. **Servidores VPS Dedicados (Ej. DigitalOcean / Hetzner):** La solución más recomendada si el cine expande su volumen transaccional, requiriendo acceso por terminal Linux y un servidor NGINX para coordinar las peticiones.

12. Bibliografía y Recursos Consultados

Para la consecución de la arquitectura, diseño UI/UX y lógicas empresariales del presente proyecto "Screenbites Cinema", se han empleado diversas fuentes de documentación oficial y foros de ingeniería de software para la resolución de conflictos técnicos y la escritura limpia de código:

- **Laravel Framework Documentation (v.10.x / v.11.x):** Referencia principal y obligatoria para el diseño de la estructura MVC, el manejo seguro de enrutamientos HTTP web, consultas avanzadas en *Eloquent ORM*, Seeders de datos demográficos y la aplicación de validaciones.
<https://laravel.com/docs>
- **MDN Web Docs (Mozilla):** Consulta exhaustiva de especificaciones técnicas orientadas al Front-End; desde animaciones CSS complejas mediante Keyframes (usadas en los eventos especiales del cine) hasta el manejo de Event Listeners asíncronos en el DOM con Vanilla JavaScript y el uso seguro de la API `localStorage` del navegador.
<https://developer.mozilla.org/>
- **Carbon Documentation (PHP API Extension):** Empleado activamente en los Controladores para la manipulación horaria, parseo (formateo) de fechas locales de sesiones, algoritmos dinámicos de sincronización automática de días y la evaluación condicional en tiempo real de los tickets activos vs expirados.
<https://carbon.nesbot.com/docs/>
- **Stack Overflow Community:** Foros de consulta global para la resolución de excepciones concretas del Framework (como la depuración de *RouteNotFoundException*, *QueryException 1146* en las migraciones SQL), así como para la integración precisa de variables de servidor (PHP) en scripts del cliente (JavaScript) usando directivas Blade.
- **Inteligencia Artificial Generativa (LLMs):** Empleada como apoyo técnico estructurado para el *debugging* de errores de rutas, iteración rápida de código (expresiones regulares y algoritmos de moderación en bases de datos relacionales). Ha sido implementada bajo una política de supervisión lógica, conociendo y entendiendo el comportamiento interno de cada fragmento antes de su integración al núcleo de Screenbites, y para la ideación complementaria de los gráficos del proyecto.